Trabajo Práctico - Unidad 5

Administración y seguridad de Food Store: roles, auditoría, incidentes y protección de datos personales

Base de Datos II - Estudiante: Mateo Algañaraz - Repositorio MateoAlganaraz/BaseDeDatos2, carpeta **tp7/**

Este PDF resume la resolución. Los scripts, la especificación, los logs, las respuestas completas de la IA y la BITACORA.md están en **tp7/**, con un commit por parte. Todo se ejecutó solo sobre la base de laboratorio propia **food_store_tp7_lab** (PostgreSQL 16.15, localhost). **Herramientas:** OpenCode y Kiro no estaban disponibles en el entorno de resolución. Las consultas a la IA se hicieron con instancias separadas de Claude sin acceso al repositorio, y la especificación se escribió en formato Kiro. Los prompts literales están en tp7/docs/ia/.

Parte A - Roles y permisos de mínimo privilegio

La spec (requirements, design y tasks) se escribió antes que **roles.sql**. Separa propiedad, privilegios e identidades. **food_store_owner** (sin login) es dueño de todo. Los roles de grupo son rol_app_lectura, rol_app_escritura, rol_soporte, rol_reportes y rol_auditor (este último es adicional). Los logins son **app_web** (miembro de lectura y escritura) y **admin_datos** (miembro de food_store_owner con INHERIT FALSE: necesita SET ROLE). Permisos de columna: rol_app_lectura lee usuario sin password_hash ni teléfono; soporte, todo menos el hash; la app actualiza solo producto.stock y las columnas editables de cliente, pedido y detalle. Las tablas futuras nacen sin privilegios y las funciones futuras sin EXECUTE para PUBLIC (ALTER DEFAULT PRIVILEGES).

```
== 3. Esperado vs. real (ni más ni menos: 0 y 0) ==
               control                | filas
--------------------------------------+-------
 esperado EXCEPT real (falta otorgar) |     0
 real EXCEPT esperado (sobra)         |     0
 privilegios esperados                |    42
 privilegios reales                   |    42
(4 rows)
```

Verificación esperado vs. real (role_table_grants + privilegios de columna y de función): 42 = 42, 0 diferencias. Además, \du y 31/31 pruebas funcionales con SET ROLE (docs/verificacion_permisos.txt).

Auditoría de roles.sql con IA: se aceptó quitar el SELECT automático sobre tablas futuras (un error de la v1), el UPDATE por columna, CONNECT para los roles de grupo, ROUTINES, los límites de sesión de app_web y la prohibición de que soporte resetee cuentas ADMIN. **Se rechazaron por ser más amplias de lo necesario** GRANT rol_app_lectura TO rol_app_escritura y UPDATE (intentos_fallidos, bloqueado_hasta) para soporte. También se detectaron errores de la IA: columnas inexistentes y una restricción CHECK que ya existía.

Parte B - Auditoría

Parámetro	Antes	Después
log_connections / log_disconnections	off / off	on / on
log_statement	none	mod (y all solo para app_web)
log_line_prefix	%m [%p] %q%u@%d	%m [%p] %q%u@%d %h app=%a
log_timezone	Etc/UTC	America/Argentina/Buenos_Aires

```
09:41:02.165 -03 [687] root [local] app=[unknown] LOG:  connection authorized: user=root database=food_store_tp7_lab application_name=psql
09:41:02.175 -03 [688] app_web 127.0.0.1 app=[unknown] LOG:  connection authorized: user=app_web database=food_store_tp7_lab
application_name=foodstore_web SSL enabled (protocol=TLSv1.3, cipher=TLS_AES_256_GCM_SHA384, bits=256)
09:41:02.176 -03 [688] app_web 127.0.0.1 app=foodstore_web LOG:  statement: SELECT current_user, inet_client_addr() AS origen;
09:41:02.176 -03 [688] app_web 127.0.0.1 app=foodstore_web LOG:  statement: SELECT id_producto, nombre, precio_lista, stock
09:41:02.177 -03 [688] app_web 127.0.0.1 app=foodstore_web LOG:  statement: WITH nuevo AS (
09:41:02.179 -03 [688] app_web 127.0.0.1 app=foodstore_web LOG:  statement: UPDATE producto SET stock = stock - 2
09:41:02.180 -03 [688] app_web 127.0.0.1 app=foodstore_web LOG:  statement: SELECT fn_autenticar('nadie.tp7audit@correo.test', 'TP7-AUDIT-clave-
erronea') AS autenticado;
09:41:02.191 -03 [689] admin_datos 127.0.0.1 app=[unknown] LOG:  connection authorized: user=admin_datos database=food_store_tp7_lab
application_name=psql_admin SSL enabled (protocol=TLSv1.3, cipher=TLS_AES_256_GCM_SHA384, bits=256)
09:41:02.193 -03 [689] admin_datos 127.0.0.1 app=psql_admin ERROR:  permission denied for schema public at character 22
09:41:02.193 -03 [689] admin_datos 127.0.0.1 app=psql_admin LOG:  statement: COMMENT ON TABLE producto IS 'Catálogo de Food Store (comentario
agregado en la prueba TP7-AUDIT)';
09:41:02.200 -03 [690] app_web 127.0.0.1 app=[unknown] FATAL:  password authentication failed for user "app_web"
```

Fragmento de docs/log_auditoria_prueba.txt (líneas de sentencias, errores y autorizaciones): lectura, inserción, actualización, fn_autenticar fallido, admin_datos sin y con SET ROLE, login fallido.

Útil: cuándo (%m), quién (%u e identidad SCRAM), desde dónde (%h), con qué aplicación (%a), qué sentencia y qué falló (ERROR/FATAL con la regla de pg_hba). **Fuera de alcance:** filas leídas o modificadas, lo que pasa dentro de funciones, la identidad efectiva tras SET ROLE y las lecturas de los demás roles. pgaudit agregaría auditoría por clase y por objeto. **El log es sensible:** contiene emails y las contraseñas pasadas a fn_autenticar

en claro. Debería leerlo solo la cuenta postgres y un grupo de auditoría reducido (archivo 0640 postgres:adm, log_file_mode 0600); en el motor, solo superusuarios o pg_read_server_files.

Parte C - Simulacro de incidente

Como app_web desde sqlcli: 4 fallos y 1 acierto contra fn_autenticar sobre la cuenta 17 (contraseña débil). SELECT * FROM usuario denegado por el hash, pero la lectura por columnas devolvió 2.000 filas. Los cinco intentos de escalamiento fallaron: GRANT admin_datos (x2), SET ROLE admin_datos, fn_resetear_password y DELETE de la auditoría.

```
09:42:57.206 -03 [848] app_web ip_origen_1 app=sqlcli LOG:  statement: SELECT current_user, inet_client_addr(), current_setting('application_name');
09:42:57.206 -03 [848] app_web ip_origen_1 app=sqlcli LOG:  statement: SELECT fn_autenticar('usuario_1@ejemplo.invalid', '[CLAVE_REDACTADA]');
09:42:57.212 -03 [848] app_web ip_origen_1 app=sqlcli LOG:  fn_autenticar: intento fallido id_usuario=17 intentos=1
09:42:57.212 -03 [848] app_web ip_origen_1 app=sqlcli LOG:  statement: SELECT fn_autenticar('usuario_1@ejemplo.invalid', '[CLAVE_REDACTADA]');
09:42:57.215 -03 [848] app_web ip_origen_1 app=sqlcli LOG:  fn_autenticar: intento fallido id_usuario=17 intentos=2
09:42:57.216 -03 [848] app_web ip_origen_1 app=sqlcli LOG:  statement: SELECT fn_autenticar('usuario_1@ejemplo.invalid', '[CLAVE_REDACTADA]');
09:42:57.219 -03 [848] app_web ip_origen_1 app=sqlcli LOG:  fn_autenticar: intento fallido id_usuario=17 intentos=3
09:42:57.219 -03 [848] app_web ip_origen_1 app=sqlcli LOG:  statement: SELECT fn_autenticar('usuario_1@ejemplo.invalid', '[CLAVE_REDACTADA]');
09:42:57.223 -03 [848] app_web ip_origen_1 app=sqlcli LOG:  fn_autenticar: intento fallido id_usuario=17 intentos=4
09:42:57.223 -03 [848] app_web ip_origen_1 app=sqlcli LOG:  statement: SELECT fn_autenticar('usuario_1@ejemplo.invalid', '[CLAVE_REDACTADA]');
09:42:57.226 -03 [848] app_web ip_origen_1 app=sqlcli LOG:  fn_autenticar: inicio de sesion correcto id_usuario=17
09:42:57.227 -03 [848] app_web ip_origen_1 app=sqlcli LOG:  statement: SELECT * FROM usuario;
09:42:57.227 -03 [848] app_web ip_origen_1 app=sqlcli ERROR:  permission denied for table usuario
09:42:57.227 -03 [848] app_web ip_origen_1 app=sqlcli LOG:  statement: SELECT id_usuario, nombre, apellido, email, rol, activo FROM usuario;
09:42:57.231 -03 [848] app_web ip_origen_1 app=sqlcli LOG:  statement: SELECT count(*) FROM usuario;
09:42:57.232 -03 [848] app_web ip_origen_1 app=sqlcli LOG:  statement: GRANT admin_datos TO rol_reportes;
09:42:57.232 -03 [848] app_web ip_origen_1 app=sqlcli ERROR:  permission denied to grant role "admin_datos"
09:42:57.232 -03 [848] app_web ip_origen_1 app=sqlcli LOG:  statement: GRANT admin_datos TO app_web;
09:42:57.232 -03 [848] app_web ip_origen_1 app=sqlcli ERROR:  permission denied to grant role "admin_datos"
09:42:57.232 -03 [848] app_web ip_origen_1 app=sqlcli LOG:  statement: SET ROLE admin_datos;
09:42:57.232 -03 [848] app_web ip_origen_1 app=sqlcli ERROR:  permission denied to set role "admin_datos"
09:42:57.232 -03 [848] app_web ip_origen_1 app=sqlcli LOG:  statement: SELECT fn_resetear_password(1, '[CLAVE_REDACTADA]');
09:42:57.232 -03 [848] app_web ip_origen_1 app=sqlcli ERROR:  permission denied for function fn_resetear_password
09:42:57.232 -03 [848] app_web ip_origen_1 app=sqlcli LOG:  statement: DELETE FROM auditoria_acceso;
09:42:57.232 -03 [848] app_web ip_origen_1 app=sqlcli ERROR:  permission denied for table auditoria_acceso
```

Log anonimizado (docs/log_simulacro_anonimizado.txt), filtrado a sentencias, errores y mensajes de fn_autenticar. Es lo único que recibió la IA.

```
== C.2.1 Membresías en admin_datos (esperado: ninguna) ==
 miembro | rol
---------+-----
(0 rows)

== C.2.2 Membresías de app_web y rol_reportes (esperado: sin cambios respecto de roles.sql) ==
 miembro |   es_miembro_de   | inherit_option | set_option
---------+-------------------+----------------+------------
 app_web | rol_app_escritura | t              | t
 app_web | rol_app_lectura   | t              | t
(2 rows)

== C.2.3 La cuenta 1 no recibió un reseteo de contraseña (esperado: 0 eventos RESET_PASSWORD) ==
 resets_cuenta_1
-----------------
               0
(1 row)
```

Verificación como administrador: admin_datos sin miembros, membresías sin cambios y ningún reseteo de contraseña. El escalamiento falló.

Contraste con la IA: acertó la secuencia completa, los motivos de cada rechazo y que la lectura parcial de usuario era lo más grave. **No sustentado por el log:** que el usuario 1 fuera administrador (es OPERADOR), que el bloqueo no se aplicara en la sesión (el umbral es 5) y el volumen leído dado como hecho. **Omisiones:** el origen real era 127.0.0.1, así que bloquear la IP no sirve e indica un acceso al host; tampoco mencionó el corte de servicio al rotar app_web ni la Ley 25.326. **Decisión del equipo**, tomada con evidencia sin anonimizar: terminar sesiones de app_web ajenas a la tienda, rotar la credencial coordinada con el despliegue, restringir pg_hba e investigar el acceso al host, resetear la cuenta 17, evaluar la notificación por la exposición de 2.000 emails y corregir el diseño (perfil propio en lugar del SELECT por columnas, bloqueo a 3 con retardo, pgaudit). No se revierte ningún rol, porque nada cambió.

Parte D - Anonimización y equivalencia

usuario_anon: seudónimo determinista por md5(sal ‖ id), nombres y emails sintéticos, teléfono y hash suprimidos, fecha generalizada a mes; solo rol_reportes la lee. Tiene 0 emails reales y 0 nombres reales. Riesgo residual documentado: hay grupos (rol, mes) de 1 persona. Las consultas de agregación por rol y mes las generó la IA a partir de una especificación precisa y se ejecutaron sin cambios:

```
== D.3.3 Equivalencia formal (esperado: 0 y 0) ==
        control         | filas
------------------------+-------
 Q_real EXCEPT Q_anon |     0
 Q_anon EXCEPT Q_real |     0
 grupos en Q_real     |    74
 grupos en Q_anon     |    74
(4 rows)

== D.3.4 Totales por rol (los mismos en ambas) ==
    rol     | usuarios_real | usuarios_anon | activos_real | activos_anon
------------+---------------+---------------+--------------+--------------
 ADMIN      |            20 |            20 |           20 |           20
 CLIENTE    |          1699 |          1699 |         1625 |         1625
 OPERADOR   |           121 |           121 |          116 |          116
 REPARTIDOR |           160 |           160 |          153 |          153
(4 rows)
```

Equivalencia: EXCEPT 0 y 0, 74 = 74 grupos y mismos totales; la tabla de los 74 grupos está en docs/verificacion_equivalencia.md. Control negativo: sin AT TIME ZONE y con TimeZone Asia/Tokyo difieren 28 grupos.

Nunca hacia una IA sin anonimizar: nombre, email, teléfono y dirección de usuarios y clientes con su historial; password_hash, contraseñas del log y credenciales; y los logs o la auditoria_acceso sin procesar, que combinan email, IP y horario.

Parte E - Bitácora e historial

BITACORA.md tiene 8 registros (prompt, respuesta resumida y validación humana) y explica las herramientas usadas. Caso más significativo: la reconstrucción del incidente, porque sus afirmaciones sin sustento habrían orientado mal la contención. Se detectó exigiendo para cada afirmación una línea de evidencia en el log original, el script o la base. Historial: un commit por parte (A, B, C, D, E) con mensajes descriptivos, y una nota sobre qué se haría distinto.